Phase 1 - Part 3

Layered Architecture Structure

In Python FastAPI application, a layered architecture is generally Preferred over a strict MVC Pattern to achieve better Seperation of testability & Scalability.

Let's Understand MVC Architecture then we deep dive into Why need Layered Architecture.

MVC Architecture

MVC (model, view, controller) architecture is design Pattern used to build web application.

BeFore MVC

Befor application started following MVC, the approach was used in which User interface code, the logic and database were all coded on the same file. This makes application heavy, slow, and complicates. and hard to manage and debug.

MVC is used to break up a large application into Smaller Sections. Each Section of the Project is designed to illustrate its own Function.

Most Popular framework Follow mvc

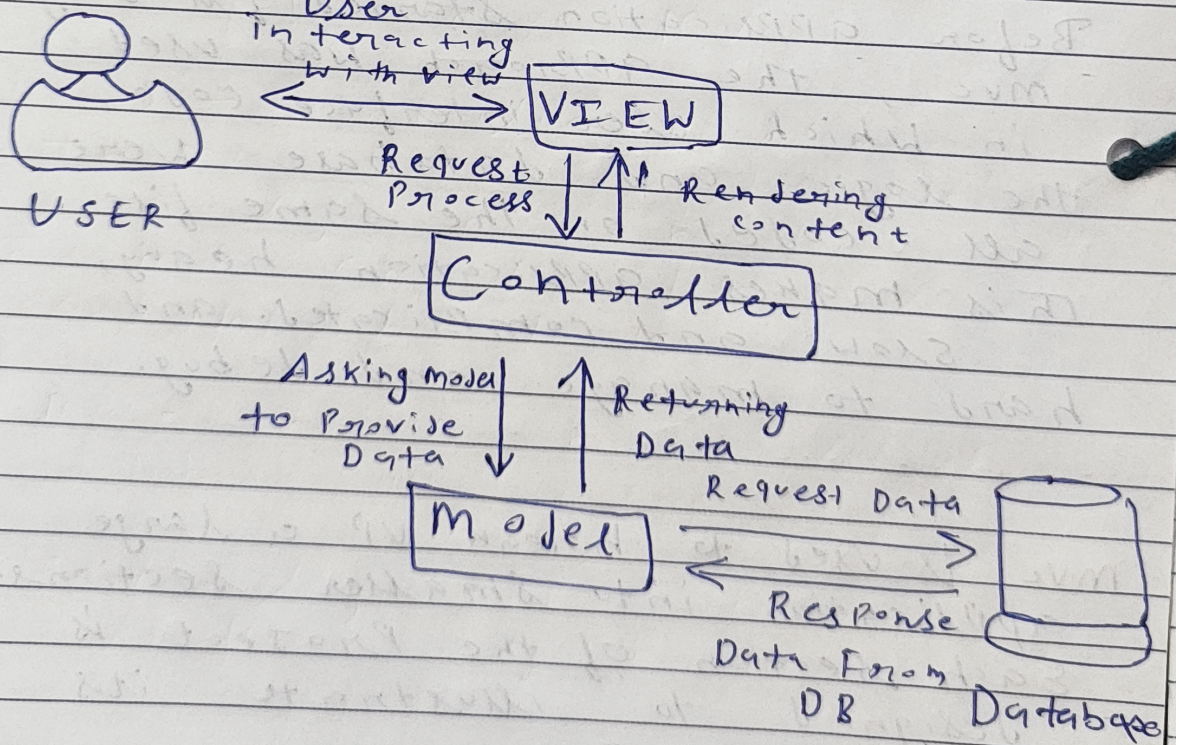
Component of mvc

① View - View represent the way that data is Presented in the application. This is where the User interacts.

② Controller - It handle User interaction. User input is interpreted by controller, causing model and view to change based on information it receives.

③ Model - It store data logic

Working of mvc



Example - https://www.interviewbit.com/online-python-compiler

1. The request first goes to the Router of the Root of the application (https://interviewbit.com). Then the root router will understand the URL with the compi controller to transfer the request: In this case, it is (online-python-compiler).

2. Controller named with (online-python-compiler) get the request. And then this controller passes the request to its appropriate Action.

3. NOW action identifies the request type and the name associated with that and processes the request and send appropriate result to the view.

4. Action Process request, in the case there is requirement of the database interaction then the particular action interact with the model requesting data from database. And return to action Then the action send appropriate result to the view.

Layered Architecture

A Project Structure of a Scalable FastAPI application.

```
app/
├── api/                # API Layer
│   ├── v1/             # versioning API
│   │   ├── endpoints/
│   │   └── api.py
├── core/               # configuration, Database, setup
├── services/           # Services layer
├── repositories        # Database Access
├── models/             # SQLAlchemy ORM model
├── schemas             # Pydantic models
├── main.py             # Application layer point
└── test.py             # Unit and integration test.
```

Why need layered Architecture if we have MVC?

1. Fat controller Problem
   In mvc, Developer often dump business logic into controller. This makes controller file massive and impossible to test.
2. mvc - controller handle Validation, database calls, business logic.
3. Service layer extract business logic. controller only handle HTTP Details.

2. Database Independence

- Mvc: model usually is the database table. if you change your database you break your model.

- Layered: application talk to interface. You can swap PostgreSQL for MongoDB only by changing Repository layer.